Software Requirements Specification

Project:	NetSentinel Platform (Real-Time Network Security Monitoring & Intrusion Detection System)
Date:	September 26, 2026 (Final Academic Release)
Version:	2.0 (Production Release & Full Pipeline Architecture) • Track: Java PBL A.Y. 2026-27
Project Guide:	Er. Ram Babu Buri (Assistant Professor, Department of Computer Science & IT)

1. Introduction

This Software Requirements Specification (SRS) establishes the architectural, functional, and non-functional specifications for **NetSentinel**, an enterprise-grade real-time Network Security Monitoring (NSM) and Intrusion Detection System (IDS). Traditional intrusion inspection platforms (e.g., Snort, Zeek) rely on platform-dependent C kernel drivers (WinPcap/Npcap), closed binary runtimes, and opaque configurations that hinder cross-platform academic deployment. NetSentinel resolves this by providing a 100% self-contained network security pipeline engineered in **Modern Java (Java 21 LTS, Spring Boot 3, Project Loom Virtual Threads)** paired with a responsive Single-Page Application (SPA) Security Operations Center (SOC) dashboard.

The platform bridges low-level packet capture and operational threat intelligence. It ingests traffic via dual modes—pure Java binary libpcap dissection and physical virtual-thread decoy honeypots—executes deterministic rule matching alongside statistical flow anomaly inference, persists immutable security events into an ACID-compliant SQLite relational database in Write-Ahead Logging (WAL) mode, and streams live telemetry to a web operations console.

2. System Overview, User Classes & RBAC Architecture

NetSentinel follows a decoupled pipeline architecture where five specialized modules communicate via frozen JSON schemas (`PacketData` and `AlertData`). The backend runtime leverages Java 21 Virtual Threads (`Thread.ofVirtual()`), zero-configuration SQLite persistence (`nsm.db`), and an HTML5/JavaScript dashboard.

2.1 User Classes

- **Security Analyst (SOC Operator):** Operational user. Monitors real-time traffic volume, inspects alert telemetry, reviews severity classifications (`[CRITICAL]`, `[HIGH]`, `[MEDIUM]`, `[LOW]`), analyzes malicious source IP frequencies, toggles Live/Pause telemetry feeds, inspects PCAP uploads, and exports threat reports (CSV/JSON).
- **Network Administrator:** Administrative user with elevated clearance. Manages detection thresholds, reviews pending user registrations, approves or rejects analyst accounts, triggers attack simulations, flushes coordinator state, and purges SQLite alerts.
- **Red-Team Evaluator / Simulated Attacker:** Generates live network probe traffic (Nmap SYN scans, hping3 floods, Hydra SSH brute force) or triggers automated single-click simulation attack scenarios against decoy listeners.

2.2 Role-Based Access Control (RBAC) Specification (Option A)

NetSentinel enforces strict database-backed Role-Based Access Control (Option A RBAC) via Spring Security:

- **Database User Directory:** Users are stored in a dedicated SQLite `users` table with BCrypt password hashing (2^{10} rounds). The system authenticates users dynamically via `DatabaseUserDetailsService` implementing Spring Security's `UserDetailsService`.
- **Self-Service Analyst Registration:** Public registration endpoint (`POST /api/auth/register`) accepts Analyst requests. Newly registered accounts default to `ROLE_ANALYST` with `status = 'PENDING'` and are blocked from login until administrative approval.
- **Team Administrator Registration Passcode:** Registration requests providing the secret team authorization passcode (configured via `nsm.security.admin-code=Creamroll`) are instantly auto-provisioned with `ROLE_ADMIN` and `status = 'APPROVED'`.
- **Admin User Approval Endpoints:** Administrators manage analyst onboarding via protected endpoints: `GET /api/auth/pending-users`, `POST /api/auth/approve-user/{id}`, and `POST /api/auth/reject-user/{id}`.
- **Endpoint Authorization Matrix:** Destructive actions (`/api/alerts/clear`), user management, and attack triggers require `ROLE_ADMIN`; telemetry observation and alert feeds permit both `ROLE_ADMIN` and `ROLE_ANALYST`.

> **Architectural Note:** All system endpoints enforce global Cross-Origin Resource Sharing (CORS) and form-based session authentication with custom handlers, permitting the dashboard to run embedded within Spring Boot or from external dev servers.

3. Functional Requirements (Core Modules)

Module 1: Packet Capture & Preprocessing (Pure Java) • Lead: Nandlal Upman

Serves as the primary data ingestion boundary, converting raw network frames and binary files into Java packet records without native C drivers.

- **Binary PCAP Dissection:** Parse standard libpcap (`.pcap`) files in pure Java, supporting both Big-Endian (`0xA1B2C3D4`) and Little-Endian (`0xD4C3B2A1`) formats, with in-place buffer positioning to ensure zero compiler warnings under Java 21.
- **Virtual-Thread Decoy Honeypots:** Bind unprivileged TCP listening sockets across decoy ports `2222` (SSH), `8088` (HTTP-Alt), `8888` (Proxy), and `9090` (Admin Console) using Project Loom virtual threads to intercept real-time network probes.
- **IPv6 Loopback Normalization:** Sanitize incoming client IP addresses, automatically translating IPv6 loopback notations (`0:0:0:0:0:0:0:1` , `::1`) into standard IPv4 `127.0.0.1` to ensure uniform rule evaluation and geolocation mapping.
- **Synthetic Attack Suite & PCAP Replay:** Provide `PacketGenerator.java` to synthesize Port Scans, SYN Floods, and Brute Force packet bursts, and accept multipart file uploads via `POST /api/attack/pcap-upload` replaying packets at 15 ms intervals.

Module 2: Rule-Based Detection Engine • Lead: Piyush Agarwal

Delivers deterministic, signature-based intrusion detection with sliding time windows and enterprise MITRE ATT&CK mappings.

- **Sliding-Window State Queues:** Maintain per-source queues via `HashMap<String, ArrayDeque<Instant>>` for $O(1)$ amortized insertion and eviction.
- **Deterministic Signatures:** Trigger alerts on: (1) **Port Scan:** ≥ 15 destination ports in 10s (conf. 0.90, MITRE T1046); (2) **SYN Flood:** ≥ 50 SYNs without ACK in 5s (conf. 0.95, MITRE T1499); (3) **SSH Brute Force:** ≥ 5 attempts on port 22 in 10s (conf. 0.85, MITRE T1110); (4) **Decoy Trap:** TCP connection to decoy ports (conf. 1.00, MITRE T1046).
- **Alert Deduplication & Reset:** Enforce a 10-second alert cooldown per `(srcIp, attackType)` to prevent flooding. Expose `reset()` to atomically flush queues and cooldown tables during state clear operations.

Module 3: Anomaly & ML Detection Engine • Lead: Mayank Jangid

Secondary behavioral detection layer analyzing statistical flow deviations that bypass static thresholds.

- **Bidirectional Flow Feature Extraction:** Aggregate raw packets into 5-second flow windows, computing packet volume, byte counts, distinct target ports, and SYN/ACK ratios. Automatically prune inactive flows older than 120s to bound JVM heap memory.
- **Shannon Port Entropy Inference:** Calculate Shannon entropy ($H = -\sum p_i \log_2 p_i$) using pre-computed $\log_2$ lookup table approximations. When traffic drops from normal baseline (3.2–4.1 bits) below 1.5 bits across concentrated ports, trigger CRITICAL anomaly alerts (conf. 0.88).
- **Volumetric Asymmetry Anomaly:** Flag asymmetric TCP flows with SYN/ACK ratio ≥ 10.0 as volumetric flood anomalies.

Module 4: Central Backend & Storage Orchestration (Spring Boot 3) • Lead: Nikhil Kumar

Central coordinator connecting capture ingestion, dual detection engines, SQLite WAL persistence, and security REST APIs.

- **Pipeline Coordination & Concurrency:** Ingest packet streams and dispatch records concurrently to Modules 2 & 3 via virtual threads; persist generated `AlertData` into SQLite (`nsm.db`) with zero lock contention (`SQLITE_BUSY = 0`).
- **Frozen REST APIs:** Expose `/api/alerts` , `/api/traffic-stats` , `/api/summary` , `/api/top-ips` , and atomic `/api/alerts/clear` .
- **Automated Integration Testing:** Maintain 18 automated JUnit 5 & MockMvc integration tests (`NsmBackendIntegrationTest.java`) covering RBAC authentication, registration, admin passcode auto-provisioning, pending approval, and attack triggers.
- **Standalone Packaging:** Package as a single executable Fat JAR (`module-4-backend-storage-1.0.0.jar`) launched via `run.bat` .

Module 5: Real-Time Web Operations Dashboard (HTML5 / JS / Chart.js) • Lead: Parikshit Vyas

Single-Page Application (SPA) delivering multi-view SOC monitoring, interactive forensic controls, and threat visualization.

- **5-View SPA Architecture:** Seamless client-side routing across: (1) *View 1: Real-Time Telemetry & ECG Rhythm Oscilloscope* (Chart.js 1Hz graph, canvas ECG oscilloscope, audio siren); (2) *View 2: Deep Packet Inspection* (PCAP drag-and-drop file upload); (3) *View 3: Attack Simulation Studio* (**horizontal flexbox layout** for Port Scan, SYN Flood, Brute Force, Replay PCAP); (4) *View 4: MITRE ATT&CK Coverage* (dynamic active vector filtering, green shield empty state, 1-click CSV/JSON exports); (5) *View 5: 3D Threat Globe* (Three.js/globe.gl ballistic arcs, local offline earth textures `earth-night.jpg` , 2D Leaflet canvas fallback).
- **Enterprise RBAC Interface:** Formal login banner (*Role-Based Access Control • Administrator-Approved Registration*), Analyst Registration Modal with Team Passcode field, and Admin User Management Modal with 1-click Approve/Reject actions.

4. Database Architecture (Relational & In-Memory Strategy)

NetSentinel employs a hybrid persistence architecture: durable SQLite relational storage in Write-Ahead Logging (WAL) mode for audit logs, alerts, and user accounts, paired with thread-safe in-memory rolling buffers for high-frequency dashboard telemetry.

Feature Category	SQLite Relational Core (`nsm.db`)	In-Memory Rolling Buffer (JVM Heap)
Tables & Types	`alerts` (forensic logs), `packets` (inspected frames), `users` (RBAC credentials).	Rolling 30s Packets/Sec, 1Hz Timestamps, Sliding Window Deques.
Schema Constraints	Relational DDL, Primary Keys, Foreign Keys, `idx_alerts_ts` (timestamp DESC).	Circular arrays, thread-safe synchronized lists, FIFO ArrayDeques.
Access Mechanism	Spring `JdbcTemplate` via `sqlite-jdbc` driver with connection pooling.	Atomic primitives (`AtomicLong`) and concurrent hash collections.
Primary Key Strategy	8-char hex UUIDs (alerts), auto-increment INT (packets, users).	In-memory circular buffer index positions (0 to 29).
Concurrency Mode	`PRAGMA journal_mode = WAL`; concurrent non-blocking reads during writes.	Lock-free atomic increments; zero cross-thread blocking.

5. Non-Functional Requirements

- **Sub-Millisecond Processing:** Binary packet dissection and rule evaluation must execute in under 0.20 ms. REST APIs must resolve `/api/traffic-stats` in under 10 ms (measured: 6.4 ms).
- **Security & Input Sanitization:** Passwords must be hashed using BCrypt (2^{10} cost factor). String inputs rendered in DOM must be escaped against XSS. Honeypots must safely terminate TCP handshakes without leaking OS banners.
- **Fault Tolerance & Engine Isolation:** Format anomalies in Module 3's ML engine must not block Module 2 rule matching or SQLite writes. In-memory queues must isolate transient DB delays.
- **Concurrency via Virtual Threads:** All socket listeners, packet dispatchers, and simulation tasks must execute on **Java 21 Project Loom Virtual Threads (`Thread.ofVirtual()`)** to handle thousands of concurrent connections with sub-1% CPU overhead.
- **Zero Native C & Air-Gapped Deployment:** The platform must run on any OS with Java 21+ without WinPcap/Npcap or external databases. Offline satellite textures ensure 100% operation without Internet connectivity.

6. Verified Performance Benchmarks & Empirical Test Results

Metric / Benchmark Parameter	Project Requirement	Empirical Measured Value	Verification Method / Status
Packet Dissection Latency	< 1.0 ms / packet	**0.08 ms**	`System.nanoTime()` over 10,000 pkts (PASSED)
Rule Evaluation Latency	< 1.0 ms / packet	**0.12 ms**	ArrayDeque sliding-window benchmark (PASSED)
ML Feature Extraction Latency	< 1.5 ms / flow	**0.22 ms**	FlowFeatureExtractor timing loop (PASSED)
Sustained Ingestion Burst	> 1,000 pkts / sec	**1,450 pkts / sec**	Multi-threaded synthetic burst test (PASSED)
SQLite WAL Write Latency	< 5.0 ms / commit	**1.15 ms**	JdbcTemplate batch commit timer (PASSED)
REST API Latency (p95)	< 15.0 ms	**6.4 ms**	MockMvc / HTTP benchmark suite (PASSED)
False Positive Rate (Benign)	< 0.05%	**0.0016%**	Campus Wi-Fi benign traffic burn-in (PASSED)
Automated Integration Tests	100% Pass Rate	**18 / 18 Tests Passed**	Maven Reactor `mvn test` (BUILD SUCCESS)